

# Test Solutions

Language: Java | 8 of 10 questions attempted

## Q1. Product of Array Except Self

### Approach:

Compute prefix products and suffix products in two passes, multiplying them together without using division.

### Java Solution:

```
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] answer = new int[n];

    // answer[i] holds product of all elements to the left of i
    answer[0] = 1;
    for (int i = 1; i < n; i++) {
        answer[i] = answer[i - 1] * nums[i - 1];
    }

    // Sweep from the right, multiplying in the product of all
    // elements to the right of i
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        answer[i] *= right;
        right *= nums[i];
    }
    return answer;
}
```

### Complexity:

Time:  $O(n)$  Space:  $O(1)$  extra (output array not counted)

## Q2. Longest Consecutive Sequence

### Approach:

Put all numbers in a HashSet. Only start counting a sequence from a number that has no predecessor (num-1 not in set), then walk forward while consecutive numbers exist. Every number is visited at most twice overall.

### Java Solution:

```
public int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int n : nums) set.add(n);

    int longest = 0;
    for (int n : set) {
        // only start a sequence at its true beginning
        if (!set.contains(n - 1)) {
            int length = 1;
            int cur = n;
            while (set.contains(cur + 1)) {
                cur++;
                length++;
            }
            longest = Math.max(longest, length);
        }
    }
}
```

```
        return longest;  
    }
```

**Complexity:**

Time:  $O(n)$  Space:  $O(n)$

### Q3. 3Sum

#### Approach:

Sort the array. Fix one element, then use a two-pointer scan on the remaining sorted range to find pairs that sum to the negation of the fixed element. Skip duplicate values at each position to avoid duplicate triplets.

#### Java Solution:

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> result = new ArrayList<>();
    int n = nums.length;

    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // skip dup anchors

        int left = i + 1, right = n - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++;
                right--;
            } else if (sum < 0) {
                left++;
            } else {
                right--;
            }
        }
    }
    return result;
}
```

#### Complexity:

Time:  $O(n^2)$  Space:  $O(1)$  extra (excluding output / sort)

### Q4. Daily Temperatures

#### Approach:

Use a monotonic decreasing stack of indices. When the current temperature is warmer than the temperature at the index on top of the stack, pop it and record the day difference as the answer for that index.

#### Java Solution:

```
public int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] answer = new int[n];
    Deque<Integer> stack = new ArrayDeque<>(); // stores indices

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
            int idx = stack.pop();
            answer[idx] = i - idx;
        }
        stack.push(i);
    }
    return answer;
}
```

**Complexity:**

Time:  $O(n)$    Space:  $O(n)$

## Q5. Kth Largest Element in an Array

### Approach:

Maintain a min-heap of size k while scanning the array. Once the heap exceeds size k, pop the smallest. After processing everything, the top of the heap is the k-th largest element.

### Java Solution:

```
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }
    return minHeap.peek();
}
```

### Complexity:

Time:  $O(n \log k)$    Space:  $O(k)$

## Q7. Course Schedule II

### Approach:

Build a directed graph from prerequisites and compute in-degrees. Run Kahn's BFS topological sort: repeatedly remove nodes with in-degree 0. If all courses get ordered, return the order; otherwise a cycle exists, so return an empty array.

### Java Solution:

```
public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());

    int[] indegree = new int[numCourses];
    for (int[] pre : prerequisites) {
        int course = pre[0], prereq = pre[1];
        graph.get(prereq).add(course);
        indegree[course]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) queue.offer(i);
    }

    int[] order = new int[numCourses];
    int idx = 0;
    while (!queue.isEmpty()) {
        int course = queue.poll();
        order[idx++] = course;
        for (int next : graph.get(course)) {
            if (--indegree[next] == 0) queue.offer(next);
        }
    }

    return idx == numCourses ? order : new int[0];
}
```

**Complexity:**

Time:  $O(V + E)$    Space:  $O(V + E)$

## Q8. Lowest Common Ancestor of a Binary Tree

### Approach:

Recurse down both subtrees. If the current node is null or matches p or q, return it. If both left and right recursive calls return non-null, the current node is the LCA; otherwise propagate whichever side found a match.

### Java Solution:

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) {
        return root;
    }

    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    if (left != null && right != null) return root; // p and q split here
    return left != null ? left : right;
}
```

### Complexity:

Time:  $O(n)$  Space:  $O(h)$  recursion stack,  $h$  = tree height

## Q10. Longest Increasing Subsequence

### Approach:

Maintain an array 'tails' where tails[i] is the smallest possible tail value of an increasing subsequence of length i+1. For each number, binary search for its insertion position in tails and overwrite it (patience sorting). The final size of tails is the LIS length.

### Java Solution:

```
public int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length];
    int size = 0;

    for (int num : nums) {
        int lo = 0, hi = size;
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            if (tails[mid] < num) lo = mid + 1;
            else hi = mid;
        }
        tails[lo] = num;
        if (lo == size) size++;
    }
    return size;
}
```

### Complexity:

Time:  $O(n \log n)$  Space:  $O(n)$